

Software Engineering in the *Age of AI*.

*How the practice of building
systems changes when the systems
themselves stop being deterministic.*

Today's arc.

- 01 What is software engineering vs software programmer
- 02 Deterministic vs. probabilistic systems.
- 03 Foundations of modern language models.
- 04 Practical implications for engineering practice.
- 05 How the software development changes when the machine writes code.

Engineering is not just *programming*.

WHAT PROGRAMMING IS

One part of the job.

A bounded act. Hold the problem in your head, write the solution, make the tests pass. The scope is a session.

WHAT ENGINEERING IS

Keeping systems *working*.

Requirements, design, validation, change. Code is one ingredient. The system is the outcome. The scope is years.

WHAT
CHANGED

BEFORE

CODING

...

NOW

CODING

EVERYTHING ELSE

Three challenges that never go away.

I.

Complexity.

Every feature adds state. State compounds. The system eventually exceeds what any one person can hold in mind.

II.

Change.

Requirements shift. Libraries update. Users discover new needs. The code you wrote on Tuesday is wrong by Friday.

III.

Correctness.

Not at one moment — over time. Green tests today are no guarantee of anything tomorrow.

Deterministic vs. Probabilistic Systems.

The shift that motivates everything else we'll cover today.

The deterministic contract.

*Same input → same output.
Always. For any input. Forever.*

- Logic is explicitly authored by the developer.
- Behavior is predictable and fully traceable.
- Bugs have a cause you can point at.



The same x. The same y. Every time.

The probabilistic contract.

*Same input → a **sample** from a distribution.
Usually right. Sometimes surprising.*

- Output is drawn from a learned probability distribution.
- The same prompt can yield different answers.
- Behavior depends on context, phrasing, temperature.



Same x. Different y. Both plausible.

Side-by-side.

DIMENSION	DETERMINISTIC	PROBABILISTIC
<i>Output</i>	Computed by rule.	Sampled from distribution.
<i>Repeatability</i>	Identical across runs.	Varies; seed- and context-dependent.
<i>Origin of logic</i>	Written by a developer.	Learned from training data.
<i>Failure mode</i>	Crashes, wrong answers you can trace.	Confident answers that happen to be wrong.
<i>Debugging</i>	Read the code.	Characterize the behavior.

THE DISTINCTION TO HOLD ONTO

Rule-based *execution*

— VS —

Pattern-based *generation*

Foundations of modern *language* *models.*

*Eight quick moves from "what is a language model?" to
"what breaks in production."*

A language model is a *probability engine*.

It estimates the likelihood of a *token* appearing within a sequence.

Tokens are the atomic units — words, subwords, characters. The model doesn't "know" anything; it learns statistical patterns and turns them into powerful internal representations.

WORKED EXAMPLE

"When I hear rain on my roof, I ____"

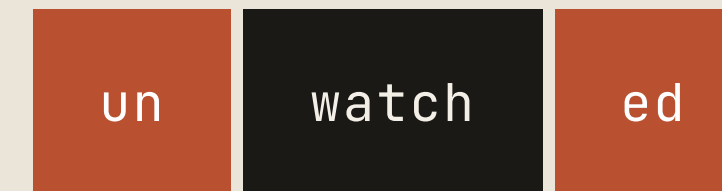
cook soup		9.4%
warm a kettle		5.2%
cower		3.6%

The atomic unit: *subwords*.

Modern models tokenize by chunks that carry *semantic meaning*.

- Roots, prefixes, suffixes transfer across unseen words.
- One English token $\approx$ 4 characters $\approx$ $\frac{3}{4}$ of a word.
- The same trick now applies to images and audio.

"UNWATCHED"



PREFIX ROOT SUFFIX

Meaning composes from pieces the model has seen before.

The problem of *context*.

N - GRAMS

Too shallow.

A 3-gram only sees the last two words. It can't know if "orange" is a fruit or a color, a name or an adjective.

orange

is

?

RNNS & LSTMS

Better, but forgetful.

Process one token at a time, gradually building context. The *vanishing gradient* means the opening of a long passage fades before the end arrives.

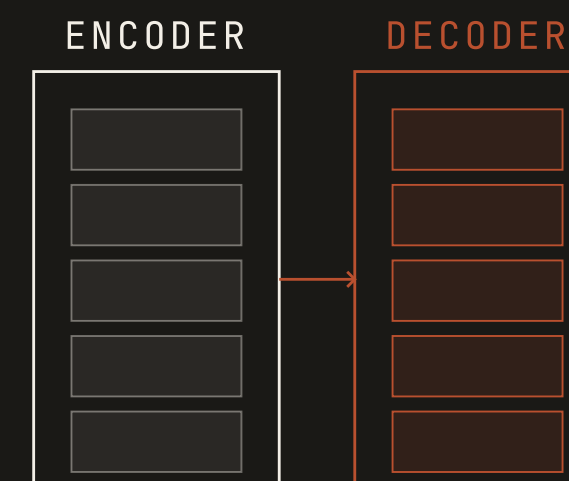
Like watching Act 3 without remembering Act 1.

The breakthrough: *transformers*.

*Every token sees every other token — **at once**.*

- Encoder builds an intermediate representation.
- Decoder generates text from it.
- Trained on *masked prediction* — guess the hidden token, at scale.

ARCHITECTURE, SCHEMATIC



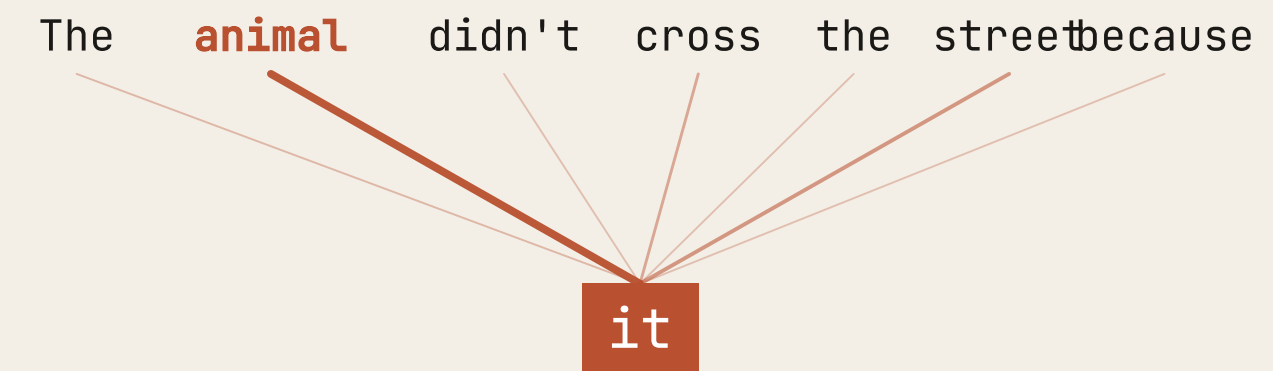
Billions of parameters. Trained on masked prediction. No recurrence.

Self-attention solves ambiguity.

*For every token, the model asks:
"how much does every other
token change my meaning?"*

Stacked attention heads learn thematic links, sentiment, and coreference across the whole input. This is how meaning gets built.

"THE ANIMAL DIDN'T CROSS THE STREET BECAUSE
IT WAS TOO TIRED."

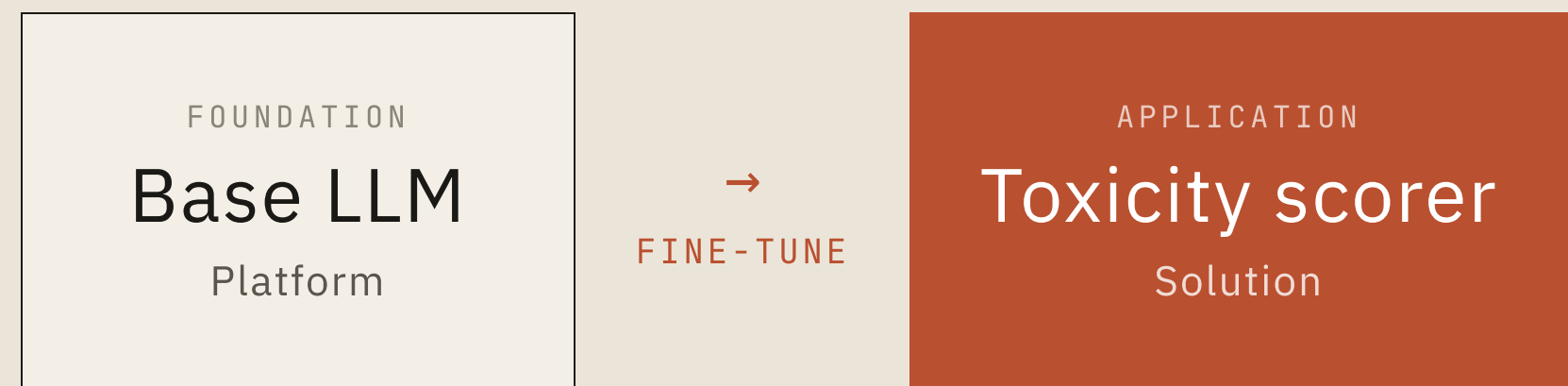


*"It" attends most strongly to "animal." Change "tired" to "wide" and
attention shifts to "street."*

Foundation models are *platforms*, not solutions.

A base LLM knows grammar and idiom. It doesn't, by itself, do *your* task.

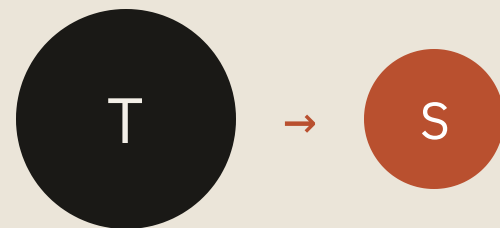
- Fine-tune on task-specific examples — often just hundreds.
- Parameter-efficient tuning updates only a *subset* of weights.
- You get specialization without retraining the world.



Making it *cheap enough to ship.*

DISTILLATION

Teacher → student.



A large teacher trains a smaller student. Faster and cheaper; some accuracy traded away.

OFFLINE INFERENCE

Pre-compute. Cache.

When the answer doesn't have to be live, run the model once and serve the cache. Google does this for query synonyms at search scale.

Latency you don't pay for is the cheapest latency.

Engineering realities: *hallucination, bias, cost.*

REALITY 01

Hallucination.

Probabilistic means plausible-sounding wins when the facts are uncertain. Confident, fluent, wrong.

REALITY 02

Bias.

The model inherits the biases of its training data — and any distillation passes them down.

REALITY 03

Cost.

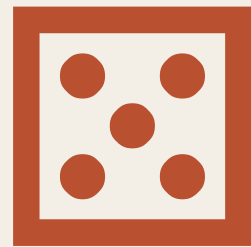
Training takes months and megawatts. Inference still costs real money at real scale.

Your job isn't just to prompt — it's to build the validation and oversight that catches each of these in turn.

Practical implications for *engineering*.

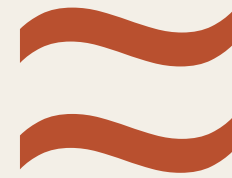
What changes about how you build, test, and ship when the substrate samples from a distribution.

Three shifts in behavior.



NON-DETERMINISM

Same call, different output. Plan for variance; don't debug it away.



PHRASING SENSITIVITY

A small prompt edit can swing behavior disproportionately.



NO CODE TO READ

The logic lives in weights. There is no branch to step through.

Common failure modes.

TYPE 01

Plausible but wrong.

The output reads correct, carries the right shape and tone, and is factually incorrect. The hardest class to catch.

TYPE 03

Stale knowledge.

Answers reflect the state of the world at training time, not today. Outdated, sometimes confidently so.

TYPE 02

Edge-case collapse.

Handles 95% of inputs gracefully. Fails on the long tail — precisely the inputs simple tests don't generate.

TYPE 04

Overgeneralization.

Given incomplete context, the model fills in the plausible gaps — and commits to them as if they

What this demands of the engineer.

SPECIFICATION

Precise, explicit, written down. The prompt is a contract; ambiguity is a bug.

VALIDATION

Stronger than unit tests. Behavioral suites that sample the distribution, not just check one path.

EDGE CASES

Systematically generated. Assumed to exist. Actively hunted, not discovered in production.

OVERSIGHT

Humans in the loop wherever the cost of a wrong answer is greater than the cost of a slow one.

From *tactical coder* to strategic designer.

- TACTICAL** The AI plays sergeant on the ground — making the code changes, line by line.
- STRATEGIC** You operate above it — system design, architecture, where the seams should fall.

Bad code is more expensive than ever. It blocks the AI from doing its best work, and it locks in choices the system has to keep paying for.

Build *deep modules* — hide complex AI-generated logic behind simple, stable, human-designed interfaces.

DEEP MODULE



A small surface area you own. A large interior the AI helps fill.

Four tactics for *staying in control*.

TACTIC 01

Clarify first.

Before any code is generated, command the model to interview you — until you share the same design concept.

TACTIC 02

Ubiquitous language.

A shared markdown of domain terms. The AI's words and yours must mean the same thing.

TACTIC 03

TDD as a speed limit.

Test-driven development forces the AI into small, deliberate steps. The test is the contract.

TACTIC 04

Don't outrun your headlights.

Continuous testing is the brake. Without it, entropy grows faster than you can read.

The clarify-first prompt.

```
> I have a plan for a new feature.  
Before you write any code,  
I want you to interview me thoroughly  
about every aspect of it.
```

1. Ask me one question at a time.
2. Walk down each branch of the design tree until we reach a shared understanding.
3. Do not generate code until I say "shared design concept reached."

WHY IT WORKS

- Forces a *human gate* before generation.
- Surfaces requirements you didn't know you had.
- Defends against Type 04: overgeneralization.
- Keeps you in the strategic seat.

Worth trying on your CSE 115A project.

Traditional *vs.* AI-driven SDLC.

The workflow itself is changing shape.

From *blueprint* to *mixed workflow*.

TRADITIONAL SDLC

A manual, linear roadmap.

A framework designed to align stakeholder needs with resources — via standardized, rule-based phases humans execute by hand.

AI-ASSISTED SDLC

A *partially probabilistic* framework.

Probabilistic tools show up in some phases — helpful in places, awkward in others. The shape is still being figured out.

The HITL shift: the engineer moves from primary author of every line → governor of a high-speed generative system.

Planning — *strategic intent*.

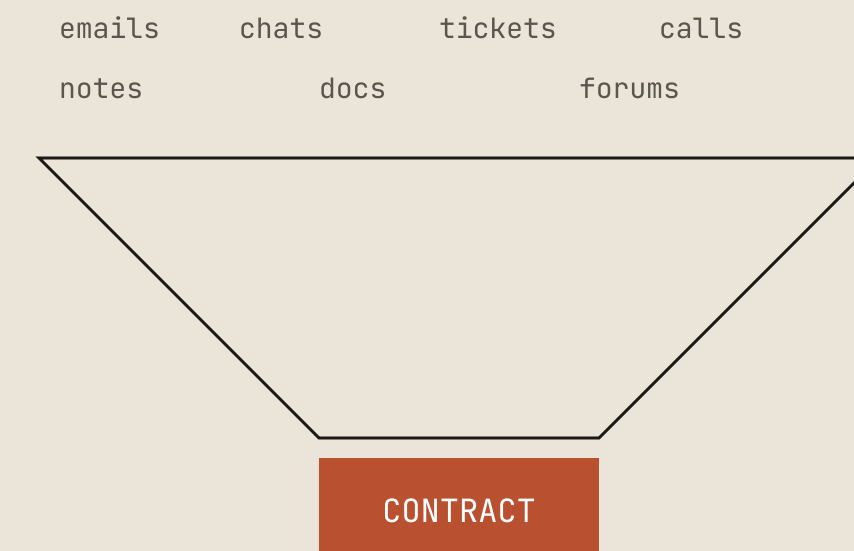
TRADITIONAL Manual interviews, brainstorming — prone to misinterpretation.

AI-DRIVEN NLP tools synthesize thousands of unstructured inputs into requirements in minutes.

HITL RESPONSIBILITY

- *Define the design concept.* Use the clarify-first protocol before any code exists.
- *Filter overgeneralization.* Catch the gaps the AI fabricated to look complete.

FUNNEL • UNSTRUCTURED → CONTRACT



The *architectural boundary*.

Spec-driven development: architectural decisions captured as executable artifacts that evolve with the project.

Rapid prototyping systems translate rough concepts into high-fidelity, testable assets.

THINGS TO WATCH

- *Interface authorship*. People design the stable contracts. The interior can be delegated.
- *Language enforcement*. Keep a shared glossary so the AI doesn't drift into jargon that contradicts business logic.

DEEP MODULE

INTERFACE • HUMAN-AUTHORED

```
{ /* implementation */  
function handle() {  
  // AI-generated  
  return compute();  
}  
  
// complex interior  
// hidden from callers  
}
```

small surface · large interior

Coding — *faster, messier.*

~59%

Reported branch-throughput lift in some studies of agentic coding. Your mileage varies.

Natural-language prompts can suggest entire functions — but speed creates its own problems.

THINGS TO WATCH

- *Prompt as contract.* Treating prompts as a precise spec helps reduce drift.
- *Entropy check.* Watch for code that passes tests but creates debt.

*Less line-by-line writing. More *reading* and *redirecting*.*

Testing — *more, not less.*

Adaptive testing: some tools focus on impacted areas and auto-repair broken tests.

One data point worth noting: main-branch success has reportedly dropped to **70.8%** in recent surveys. Generation often outpaces validation.

THINGS TO WATCH

- *Behavioral suites.* Sample the distribution of outcomes, not one "correct" path.
- *Edge-case hunting.* Chase the long-tail failures automated systems tend to skip.
- *TDD as speed limit.* If you can't write a test

BRANCH SUCCESS • MAIN

70.8%

*"Don't outrun your headlights." — CircleCI, 2024
State of Software Delivery*

Deployment — *humans still approve.*

Self-healing pipelines: some setups watch telemetry and surface failures before they escalate.

Proactive ops can correlate incidents across systems and propose fixes. 3 AM crash? The AI might draft a PR — but someone still needs to read it.

THINGS TO WATCH

- *Decision gates.* Human approvals on anything that matters.
- *Stale-knowledge monitoring.* Is the model's world the current world?

PIPELINE • 03:14 AM

▲ ALERT • build #4821 failed

test_checkout_flow • FAIL
diagnosed: regex drift on ZIP parse

✓ PR #2193 drafted • 12 lines

AWAITING • HUMAN APPROVAL

The workflow shifts.

	TRADITIONAL SDLC	AI-ASSISTED SDLC
<i>Workflow</i>	Linear — plan → build → test → ship.	Often cyclical — prompt → gen → validate → refine.
<i>Pace</i>	Measured in weeks.	Sometimes minutes, sometimes not.
<i>Human role</i>	Author — writing the logic.	More reviewing and redirecting; still authoring the hard parts.
<i>Where time goes</i>	Implementation.	More on validation and review.

Roughly, match the speed of generation with the speed of verification. If your generation loop is 10 minutes, a two-day validation loop probably isn't going to keep up.

Stay in the *driver's seat*.

WHAT STAYS WITH YOU

Worth remembering: the AI is closer to a *fancy autocomplete* than an innovator. Judgment, ethics, and messy human context don't automate well.

A REAL TRADEOFF

Some studies link heavy reliance to ~**15%** more technical debt, and possibly some skill atrophy. Worth being honest about.

If you can't *understand* the code,
you're probably closer to a *passenger*
than an engineer.

A good default for 2026: stay skeptical, stay curious, keep asking "do I actually understand this?"

What to leave with.

The *substrate* changed.
The *discipline* didn't.

*Systems still matter more than code. Specs,
validation, and edge cases matter more, not less.
Understanding model behavior is now part of the job.*

Questions for next week.

Q1 How does non-determinism change how we debug?

Q2 Which systems are better served by AI — and which aren't?

Q3 How should testing evolve for probabilistic systems?

END OF LECTURE 01

See you *Next Week.*

Read: the transformer paper.